

Robust Model Predictive Control

Learning Dynamical Systems

Lorenzo Favaron
2128830

Luca Galeazzo
2158514

Lorenzo Trubian
2125512

July 26, 2026

1 Introduction

Given the system with state $x \in \mathbb{R}^n$, output $y \in \mathbb{R}^p$, input $u \in \mathbb{R}^m$, white Gaussian noise with unit variance $v \in \mathbb{R}^{n+p}$

$$\begin{aligned}x_{t+1} &= Ax_t + Bv_t + Ku_t \\ y_t &= Cx_t + Dv_t\end{aligned}$$

defined by matrices: state-state matrix $A \in \mathbb{R}^{n \times n}$, noise-state matrix $B \in \mathbb{R}^{n \times (n+p)}$, state-output matrix $C \in \mathbb{R}^{p \times n}$, noise-output matrix $D \in \mathbb{R}^{p \times (n+p)}$, and input-state matrix $K \in \mathbb{R}^{n \times m}$.

A common assumption, also made in Section III of [1], is that the matrix $\begin{bmatrix} B \\ D \end{bmatrix}$ is square and invertible.

2 Model Predictive Control

$$\begin{aligned}x_{t+1} &= Ax_t + Ku_t \\ y_t &= Cx_t\end{aligned}$$

over the time horizon $[t, t + N]$, in the presence of polyhedral constraints on the state and input variables. Let the control sequence be $\mathbf{u}_t = (u_t, u_{t+1}, \dots, u_{t+N-1})$. The cost function is

$$J(x_t, \mathbf{u}_t, t) = \sum_{i=0}^{N-1} [\|y_{t+i} - r_{t+i}\|_Q + \|u_{t+i} - \bar{u}_{t+i}\|_R] + \|y_{t+N} - r_{t+N}\|_{P_f} \quad (1)$$

This expression can be written w.r.t. the states x

$$J(x_t, \mathbf{u}_t, t) = \sum_{i=0}^{N-1} [\|Cx_{t+i} - r_{t+i}\|_Q + \|u_{t+i} - \bar{u}_{t+i}\|_R] + \|Cx_{t+N} - r_{t+N}\|_{P_f}.$$

It possible to isolate the quadratic and linear terms in the variables x and u

$$\begin{aligned}
J(x_t, \mathbf{u}_t, t) &= \sum_{i=0}^{N-1} [\|x_{t+i}\|_{C^\top Q C} + \|u_{t+i}\|_R] + \|x_{t+N}\|_{C^\top P_f C} \\
&\quad - 2 \sum_{i=0}^{N-1} \left[(C^\top Q r_{t+i})^\top x_{t+i} + \bar{u}_{t+i}^\top R u_{t+i} \right] + (C^\top P_f r_{t+N})^\top x_{t+N} \\
&\quad + \sum_{i=0}^{N-1} [\|r_{t+i}\|_Q + \|\bar{u}_{t+i}\|_R] + \|r_{t+N}\|_{P_f}
\end{aligned}$$

where the last term of the latter expression is a constant term and can be ignored in the optimization process. Let us define

$$z = \begin{bmatrix} x_{t+1} \\ \vdots \\ x_{t+N} \\ u_t \\ \vdots \\ u_{t-1+N} \end{bmatrix}, \quad r_{x,u} = \begin{bmatrix} r_{t+1} \\ \vdots \\ r_{t+N} \\ \bar{u}_t \\ \vdots \\ \bar{u}_{t-1+N} \end{bmatrix}$$

The *non-condensed form* of the MPC problem can be written as¹

$$\begin{aligned}
\min_z J(x_t, \mathbf{u}_t, t) &= \min_z \frac{1}{2} z^\top \bar{H} z + f^\top z \\
\text{subject to } G_{eq} z &= E_{eq} x_t.
\end{aligned} \tag{2}$$

where the system evolution is encoded as equality constraint using the matrices

$$G_{eq} = \left[\begin{array}{cccc|cccc} I & & & & -K & & & \\ -A & I & & & & -K & & \\ & -A & I & & & & -K & \\ & & \ddots & \ddots & & & \ddots & \\ & & & -A & I & & & -K \end{array} \right] \quad E_{eq} = \begin{bmatrix} A \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{3}$$

Let us define the following matrices

$$\bar{C} = \left[\begin{array}{ccc|ccc} C & & & & & \\ & \ddots & & & & \\ & & C & & & \\ \hline & & & I & & \\ & & & & \ddots & \\ & & & & & I \end{array} \right], \quad H = \left[\begin{array}{ccc|ccc} Q & & & & & \\ & \ddots & & & & \\ & & P_f & & & \\ \hline & & & R & & \\ & & & & \ddots & \\ & & & & & R \end{array} \right] \tag{4}$$

which allows to compactly write the cost function of (1)

$$\bar{H} = \bar{C}^\top H \bar{C}, \quad f = -\bar{C}^\top H r_{x,u}.$$

¹This formulation is consistent with the standard form employed by quadratic programming solvers, such as MATLAB's *quadprog* and OSQP.

A natural question noticing the form of (3) is: what role do $x(k)$ and E_{eq} actually play in this problem? The key point is that the starting condition is not simply the current state, but rather the predicted free evolution based on the current state. This becomes especially important when estimators are involved: it is not strictly necessary to provide the MPC with x_t ; instead, one can supply the one-step-ahead prediction $\hat{x}_{t+1,free} = \hat{x}_{free}(t+1 | t)$ of the free evolution (the dependence on the information available at time t is omitted, since this is the most recent information accessible to the controller and thus the most accurate). In practice, this is what we have done, and the matrix E_{eq} is completely discarded, with the state-evolution constraints being reformulated accordingly as

$$G_{eq}z = x_{t+1,t+N} \quad \text{with} \quad x_{t+1,t+N} = \begin{bmatrix} \hat{x}_{t+1,free} \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

3 Robust state space estimation

In this section only, the matrix K will be assigned a meaning that differs from that of the input-state matrix. This does not pose a problem, since the matrix introduced here is used purely internally, and the lack of inputs prevents any lasting ambiguity in the notation. This convention is adopted to remain as consistent as possible with the notation commonly used in the Robust Kalman Filter literature.

3.1 Kalman Filter

$$\begin{aligned} G_t &= K_{x_{t+1}y_t} K_{y_t}^{-1} \\ &= \left(A_t V_t C_t^\top + B_t D_t^\top \right) \left(C_t V_t C_t^\top + D_t D_t^\top \right)^{-1}. \end{aligned}$$

The least-favorable covariance matrix $\tilde{K}_{x_{t+1}}$ can be evaluated as follows. Let

$$\begin{aligned} P_{t+1} &= K_{x_{t+1}} - K_{x_{t+1}y_t} K_{y_t}^{-1} K_{y_t x_{t+1}} \\ &= (A_t - G_t C_t) V_t (A_t - G_t C_t)^\top + (B_t - G_t D_t)(B_t - G_t D_t)^\top \end{aligned}$$

and

$$V_{t+1} = \tilde{K}_{x_{t+1}} - K_{x_{t+1}y_t} K_{y_t}^{-1} K_{y_t x_{t+1}}$$

denote the nominal and least-favorable conditional covariance matrices of x_{t+1} given $\mathcal{Y}_t$. Then

$$V_{t+1}^{-1} = P_{t+1}^{-1} - \lambda_t^{-1} I_n$$

where the Lagrange multiplier $\lambda_t > r(P_{t+1})$ is selected such that

$$\gamma_t(\lambda_t) = \frac{1}{2} \left[\text{tr} \left((I_n - \lambda_t^{-1} P_{t+1})^{-1} - I_n \right) + \ln \det (I_n - \lambda_t^{-1} P_{t+1}) \right] = c \quad (5)$$

A direct computation of its derivative shows that $\gamma_t(\lambda_t)$ is monotone decreasing over $(r(P_{t+1}), \infty)$ and has for range $\mathbb{R}^+$. Thus, for any divergence tolerance $c > 0$, there exists a matching Lagrange multiplier $\lambda_t > r(P_{t+1})$. Given that it is evaluated repeatedly throughout the algorithm, and that subsequent computations depending on its value exhibit high sensitivity, both numerical stability and computational efficiency must be considered when determining λ_t . In this work, we employ a

variant of the regula falsi secant method, incorporating the modifications proposed by Anderson and Björk in [2] (a general pseudo-code for this family of algorithms can be found in [3]).

Algorithm 1 Robust Kalman filter at time t .

Require: $c_t, \hat{x}_t, y_t, V_t$,

Ensure: $\hat{x}_{t+1}, V_{t+1}, G_t, \lambda_t$

- 1: $G_t \leftarrow (A_t V_t C_t^\top + B_t D_t^\top)^\top (C_t V_t C_t^\top + D_t D_t^\top)^{-1}$
 - 2: $\hat{x}_{t+1} \leftarrow A_t \hat{x}_t + G_t (y_t - C_t \hat{x}_t)$
 - 3: $P_{t+1} \leftarrow (A_t - G_t C_t) V_t (A_t - G_t C_t)^\top + (B_t - G_t D_t)(B_t - G_t D_t)^\top$
 - 4: Find λ_t such that $c_t = \gamma_\tau(P_{t+1}, \lambda_t)$
 - 5: Compute
 $V_{t+1} \leftarrow (I_n - \lambda_t^{-1} P_{t+1})^{-1} P_{t+1}$
-

3.2 Least-Favorable Model

Freely inspired by fifth section of [1], equations (86)-(92). Starting with

$$W_{T+1}^{-1} = \lambda_t^{-1} I_{n+p}$$

the backward recursion requires to compute the matrices

$$\begin{aligned} \tilde{K}_t &= \left(I_{n+p} - (B_t - G_t D_t)^\top W_{t+1}^{-1} (B_t - G_t D_t) \right)^{-1} \\ H_t &= \tilde{K}_{v_t} (B_t - G_t D_t)^\top W_{t+1}^{-1} (A_t - G_t C_t) \\ \Omega_t^{-1} &= (A_t - G_t C_t)^\top W_{t+1}^{-1} (A_t - G_t C_t) + H_t^\top \tilde{K}_t^{-1} H_t. \end{aligned}$$

The recursion continues setting the next W as:

$$W_t^{-1} = \Omega_t^{-1} + \lambda_{t-1}^{-1} I_n.$$

where L_t is an arbitrary matrix square root of $\tilde{K}_t$, i.e., $L_t L_t^\top = \tilde{K}_t$.

Freely inspired by fifth section of [1], equations (93)-(95)

$$\xi_t \triangleq \begin{bmatrix} x_t \\ e_t \end{bmatrix}$$

the least-favorable model admits a state-space representation

$$\begin{aligned} \xi_{t+1} &= \tilde{A}_t \xi_t + \tilde{B}_t \epsilon_t \\ y_t &= \tilde{C}_t \xi_t + \tilde{D}_t \epsilon_t \end{aligned}$$

with twice the dimension of the nominal state-space model, where

$$\begin{aligned} \tilde{A}_t &= \begin{bmatrix} A_t & B_t H_t \\ 0 & A_t - G_t C_t + (B_t - G_t D_t) H_t \end{bmatrix}, & \tilde{B}_t &= \begin{bmatrix} B_t \\ (B_t - G_t D_t) \end{bmatrix} L_t, \\ \tilde{C}_t &= \begin{bmatrix} C_t & D_t H_t \end{bmatrix}, & \tilde{D}_t &= D_t L_t. \end{aligned} \tag{6}$$

Algorithm 2 Least-Favorable Model for the time interval $[k, k + N]$.

Require: $c, V_k, N, N_f > N$ (default $N_f = 2N$)

Ensure: $\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t$ for $t \in [k, k + N]$

```

1:  $\triangleright$  Forward Riccati iterations at least to large enough time  $N_f$   $\triangleleft$ 
2: for  $t = 1, \dots, N_f$  do
3:    $V_{t+1}, G_t, \lambda_t \leftarrow RKF(c, V_t)$   $\triangleright$  Algorithm 1 without state-prediction (step 2)
4:  $\triangleright$  Backward sweep to compute LFM covariances  $\triangleleft$ 
5:  $W_{N_f}^{-1} \leftarrow \lambda_{N_f}^{-1} I_n$ 
6: for  $t = N_f - 1, \dots, 1$  do
7:    $\triangleright$  temporary matrices  $\triangleleft$ 
8:    $A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$ 
9:    $\triangleright$  computation of  $K^{-1}, H, L$   $\triangleleft$ 
10:   $K_t^{-1} \leftarrow I_{n+p} - B_{tmp}^\top W_t^{-1} B_{tmp}$ 
11:   $H_t \leftarrow K_t B_{tmp}^\top W_t^{-1} A_{tmp}$ 
12:   $L_t \leftarrow \text{cholesky}(K^{-1})^{-1}$ 
13:   $\triangleright$  update values for next iteration  $\triangleleft$ 
14:  if  $t = 1$  then
15:    exit cycle
16:  else
17:     $\Omega_t^{-1} \leftarrow A_{tmp}^\top W_{t+1} A_{tmp} + H_t^\top K_t^{-1} H_t$ 
18:     $W_t^{-1} \leftarrow \Omega_t^{-1} + \lambda_{t-1}^{-1} I_n$ 
19:  $\triangleright$  Forward sweep to compute the LFM matrices to  $N$   $\triangleleft$ 
20: for  $t = 1, \dots, N$  do
21:    $A_{tmp} \leftarrow A_t - G_t C, \quad B_{tmp} \leftarrow B_t - G_t D$ 
22:   Compute the matrices of LFM:
   
$$\tilde{A}_t \leftarrow \begin{bmatrix} A_t & B_t H_t \\ 0 & A_{tmp} + B_{tmp} H_t \end{bmatrix}, \quad \tilde{B}_t \leftarrow \begin{bmatrix} B_t \\ B_{tmp} \end{bmatrix} L_t, \quad \tilde{C}_t \leftarrow [C_t \quad D H_t], \quad \tilde{D}_t \leftarrow D_t L_t$$


```

4 Least favorable model for MPC

Thus, QP is now solving the MPC problem with a time-varying nominal model which corresponds to the Least-Favorable Model produced by the previous analysis. Considering the matrices (6), the system evolution constraint (3) becomes

$$G_{eq} = \left[\begin{array}{cccccc} I & & & & & \\ -\tilde{A}_2 & I & & & & \\ & -\tilde{A}_3 & I & & & \\ & & \ddots & \ddots & & \\ & & & -\tilde{A}_N & I & \end{array} \right] \begin{bmatrix} -K & & & & & \\ & -K & & & & \\ & & -K & & & \\ & & & \ddots & & \\ & & & & -K & \end{bmatrix} E_{eq} = \begin{bmatrix} \tilde{A}_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

and the matrices (4) defining the cost function are

$$\bar{C} = \left[\begin{array}{c|c} \tilde{C}_1 & \\ \hline & \tilde{C}_N \\ \hline & I \\ & \\ & \\ & \\ & I \end{array} \right], \quad \bar{H} = \bar{C}^\top \left[\begin{array}{c|c} Q & \\ \hline & P_f \\ \hline & R \\ & \\ & \\ & \\ & R \end{array} \right] \bar{C}.$$

where $\bar{H}$ and f are defined as before.

Algorithm 3 Robust MPC at time t .

Require: $c, \hat{x}_{t,free}, V_t, N$

Ensure: u_t, J_t

1: $\{\tilde{A}_i, \tilde{C}_i\}_{i \in [t, t+N]} \leftarrow LFM(c, V_t, N)$

$\triangleright$ *Algorithm 2*

2: find u_t solving the QP (2) using the time-varying matrices $G_{eq}, \bar{H}$ showed above

5 Selection of the optimal tolerance c

Up to this point, the tolerance c has been chosen a priori. The proposed approach instead allows the controller to determine c by adding an additional term to the quadratic programming (QP) problem (2) with an additional term. In this formulation, the controller autonomously balances the quality of the prediction achieved so far against the prediction adopted by the optimal control law.

$$\min_{c, u, \hat{y}, \hat{x}} \sum_{t=-L}^0 \beta^t \|y_t - \hat{y}_{t|t-1}(c)\|^2 + J(x_t, \mathbf{u}_t, t) \quad (7)$$

Once the optimal tolerance has been selected in the MPCs, using the free-evolution prediction $\hat{x}_{c_j, free}(k+1|k)$ obtained from the RKF, the optimal prediction $\hat{x}_{c_{best}}(k+1|k)$ (which already includes both the free evolution and the effect of the selected input) will serve as the initial condition for all filters in the subsequent iteration.

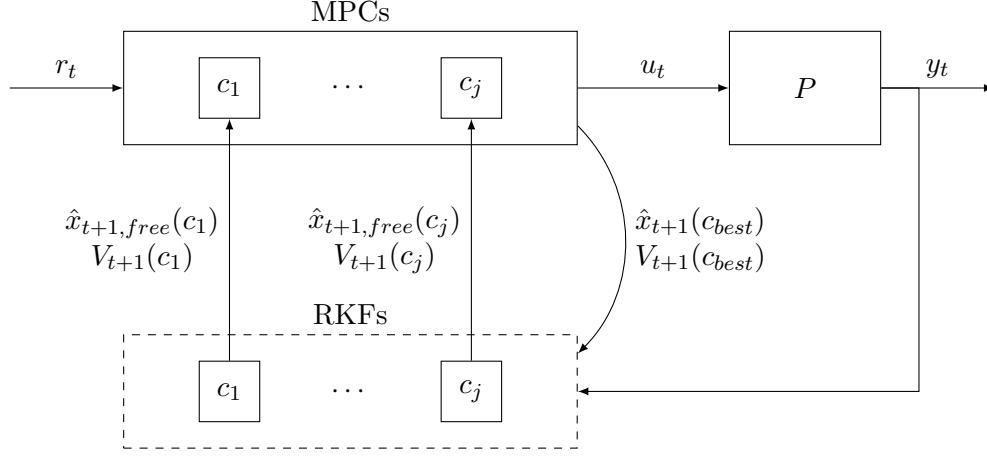


Figure 1: The dashed line emphasizes that each RKF operates independently from the others (apart from the shared initial estimate $\hat{x}$ and the measured output y), and therefore no computations are performed across filters. This independence does not hold for the MPCs: after each MPC solves its own optimization problem, an additional minimization problem is formulated that also incorporates past predictions, producing a single control input u along with an optimal tolerance and the corresponding best prediction $\hat{x}_{best}$.

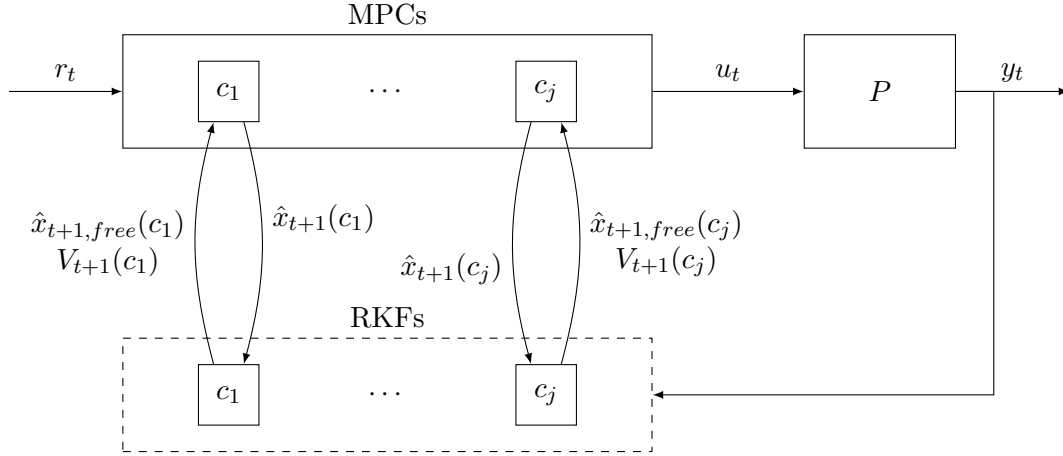


Figure 2

As illustrated by the global scheme in Figure 1, the minimization problem (7) is addressed only for a discrete set of tolerance values c . The main difficulty in tackling the full minimization problem in a single step lies in the fact that there is no closed-form expression for the Lagrange multiplier λ that satisfies equation (5).

Algorithm 4 Robust MPC with tolerance selection at time t (LFM + best prediction).

Require: $\{c_j\}_j, \hat{\xi}_t, y_t, V_t, N, \beta, L$

Ensure: $u_t, \hat{\xi}_{t+1}, V_{t+1}$

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 2 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t, N)$ 
4:    $\triangleright$  Algorithm 1 applied using the best information of the previous step
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t, y_t, V_t, \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (7)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Update prediction and covariance for the next step
13:  $\hat{\xi}_{t+1} \leftarrow \hat{\xi}_{t, \text{free}}(c_{\text{best}}) + \begin{bmatrix} K \\ 0 \end{bmatrix} u_t(c_{\text{best}})$ 
14:  $V_{t+1} \leftarrow V_{t+1}(c_{\text{best}})$ 
15:  $\triangleright$  Apply input
16:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 5 Robust MPC with tolerance selection at time t (LFM + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

Ensure: u_t

```

1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 2 to compute LFM in time interval  $[t, t + N]$ 
3:    $\{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]} \leftarrow \text{LFM}(c, V_t(c_j), N)$ 
4:    $\triangleright$  Algorithm 1 applied using the information of the previous step of the filter associated to the same  $c_j$ 
5:    $\hat{\xi}_{t+1, \text{free}}(c_j), V_{t+1}(c_j) \leftarrow \text{RKf}(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{\tilde{A}_t, \tilde{B}_t, \tilde{C}_t, \tilde{D}_t\})$ 
6:    $\triangleright$  MPC solver with LFM as nominal model
7:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow \text{MPC}(\hat{\xi}_{t+1, \text{free}}(c_j), \{\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i\}_{i \in [t, t+N]}, N)$ 
8:    $\triangleright$  Add past prediction errors to the controller cost
9:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
10:  $\triangleright$  Optimization step, problem (7)
11:  $c_{\text{best}} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
12:  $\triangleright$  Apply input
13:  $u_t \leftarrow u_t(c_{\text{best}})$ 

```

Algorithm 6 Robust MPC with tolerance selection at time t (RKF + own prediction).

Require: $\{c_j\}_j, y_t, N, \beta, L$

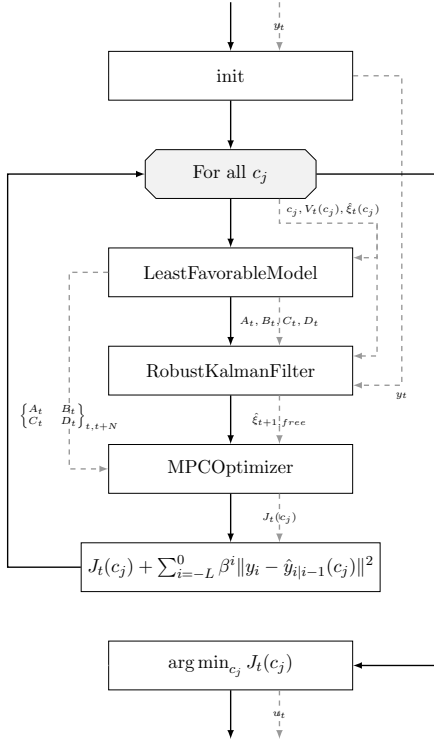
Ensure: u_t

```

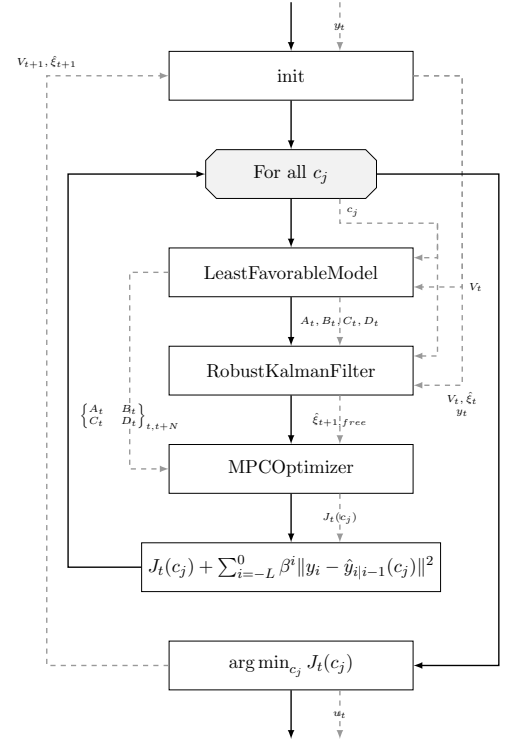
1: for all  $c_j$  do
2:    $\triangleright$  Algorithm 1 applied using the information of the previous step of the filter associated to
      the same  $c_j$   $\triangleleft$ 
3:    $\hat{\xi}_{t+1,free}(c_j), V_{t+1}(c_j) \leftarrow RKF(c_j, \hat{\xi}_t(c_j), y_t, V_t(c_j), \{A, B, C, D\})$ 
4:    $\triangleright$  MPC solver with nominal model  $\triangleleft$ 
5:    $u_t(c_j), \hat{\xi}_{t+1}(c_j), J_t(c_j) \leftarrow MPC(\hat{\xi}_{t+1,free}(c_j), \{A, B, C, D\}_{i \in [t, t+N]}, N)$ 
6:    $\triangleright$  Add past prediction errors to the controller cost  $\triangleleft$ 
7:    $J_t(c_j) \leftarrow J_t(c_j) + \sum_{i=-L}^0 \beta^i \|y_i - \hat{y}_{i|i-1}(c_j)\|^2$ 
8:  $\triangleright$  Optimization step, problem (7)  $\triangleleft$ 
9:  $c_{best} \leftarrow \arg \min_{c_j} J_t(c_j)$ 
10:  $\triangleright$  Apply input  $\triangleleft$ 
11:  $u_t \leftarrow u_t(c_{best})$ 

```

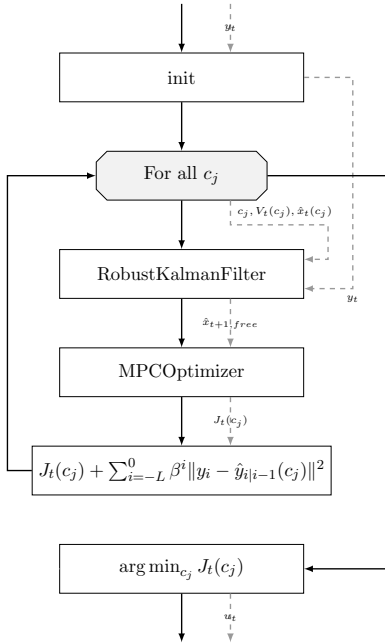
Although the two loops in Algorithm 4 could be combined into one, the code is written to emphasize the separation between the two logically distinct phases, in accordance with the block diagram shown in Figure 1. Moreover, it is worth noting that step 13 of Algorithm 4 is actually redundant, since the solution z of the minimization problem (2) already includes the prediction of the next state $\hat{x}_{t+1}$. It is presented explicitly only to illustrate as clearly as possible how the next prediction is a combination of the Robust Kalman Filter and the Model Predictive Controller.



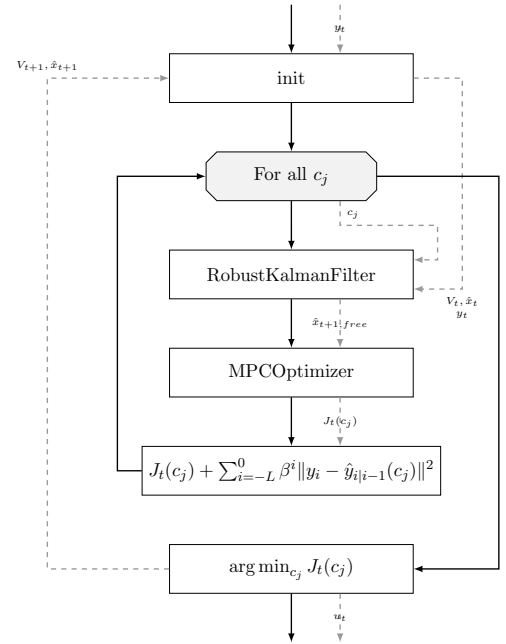
(a) LFM approach, own prediction strategy



(b) LFM approach, best prediction strategy



(c) RKF approach, own prediction strategy



(d) RKF approach, best prediction strategy. It can be dropped.

Figure 3: LFM approach, best prediction strategy

6 Teoria: Varianza dell'Input in Anello Chiuso

Il problema consiste nel valutare se l'ampiezza dell'input a disposizione sia sufficiente a mantenere il sistema a regime in presenza di disturbi stocastici, senza che gli attuatori saturino frequentemente. In presenza di un controllore Model Predictive Control (MPC) e di uno stimatore (Filtro di Kalman), il sistema opera in anello chiuso.

A regime, se i vincoli non sono attivi, il comportamento locale dell'MPC può essere approssimato dal guadagno LQR stazionario F . La legge di controllo si esprime come:

$$u_t = u_{ss} - F(\hat{x}_t - x_{ss}) \quad (8)$$

dove u_{ss} e x_{ss} sono i valori deterministici a regime e $\hat{x}_t$ è la stima dello stato. Senza perdita di generalità per l'analisi della varianza, poniamo le variabili centrate nell'origine, tale che $u_t = -F\hat{x}_t$.

Definendo l'errore di stima come $e_t = x_t - \hat{x}_t$, lo stato del sistema e lo stato stimato possono essere combinati in uno stato esteso $\xi_t = \begin{bmatrix} x_t \\ e_t \end{bmatrix}$. Sostituendo la legge di controllo nella dinamica del sistema e considerando il guadagno stazionario L del Filtro di Kalman, la dinamica in anello chiuso risulta:

$$\xi_{t+1} = \mathcal{A}_{CL}\xi_t + \mathcal{B}_{CL}v_t \quad (9)$$

con le matrici di sistema aumentato:

$$\mathcal{A}_{CL} = \begin{bmatrix} A - KF & KF \\ 0 & A - LC \end{bmatrix}, \quad \mathcal{B}_{CL} = \begin{bmatrix} B \\ B - LD \end{bmatrix} \quad (10)$$

Assumendo che v_t sia un rumore gaussiano bianco con matrice di covarianza identità, la matrice di covarianza dello stato esteso a regime, $\Sigma_\xi = \mathbb{E}[\xi_t \xi_t^T]$, si ottiene risolvendo l'equazione discreta di Lyapunov:

$$\Sigma_\xi = \mathcal{A}_{CL}\Sigma_\xi\mathcal{A}_{CL}^T + \mathcal{B}_{CL}\mathcal{B}_{CL}^T \quad (11)$$

Per proiettare questa covarianza sull'input, esprimiamo l'azione di controllo in funzione dello stato esteso:

$$u_t = \begin{bmatrix} -F & F \end{bmatrix} \xi_t = \mathcal{M}\xi_t \quad (12)$$

La covarianza dell'input è quindi:

$$\Sigma_u = \mathcal{M}\Sigma_\xi\mathcal{M}^T \quad (13)$$

Estraendo la diagonale di Σ_u otteniamo la varianza $\sigma_{u,i}^2$ di ciascun canale di input. Utilizzando la regola dei 3σ , il sistema richiederà input contenuti nell'intervallo $u_{ss,i} \pm 3\sigma_{u,i}$ per il 99.73% del tempo. Se questo intervallo eccede i limiti fisici dell'attuatore, i vincoli sono troppo stringenti.

7 Sintesi della Procedura

- **Calcolo dell'Equilibrio:** Determinare input (u_{ss}) e stato (x_{ss}) nominali per mantenere il riferimento desiderato in assenza di rumore.
- **Linearizzazione del Controllore:** Estrarre il guadagno LQR locale F associato ai pesi dell'MPC.
- **Sintesi dello Stimatore:** Calcolare il guadagno stazionario L del Filtro di Kalman, gestendo le covarianze di processo e di misura (inclusa la cross-covarianza).

- **Dinamica Congiunta:** Costruire le matrici di anello chiuso $\mathcal{A}_{CL}$ e $\mathcal{B}_{CL}$ unendo le dinamiche del plant e dell'errore di stima.
- **Risoluzione di Lyapunov:** Calcolare la covarianza dello stato esteso Σ_ξ tramite l'equazione di Lyapunov discreta (verificando preventivamente la stabilità asintotica di $\mathcal{A}_{CL}$).
- **Verifica dei Vincoli:** Estrarre la deviazione standard dell'input e testare la condizione $u_{ss,i} \pm 3\sigma_{u,i}$ rispetto ai limiti operativi fisici.

8 Codice MATLAB

Di seguito la sequenza delle istruzioni necessarie per eseguire l'analisi stocastica in MATLAB:

```
% 1. Equilibrio (Assumendo riferimento sull'output r)
uss = [A - eye(n), K; C, zeros(p,m)] \ [zeros(n,1); r];
uss = uss(n+1:end);

% 2. Calcolo dei Guadagni F ed L
F = dlqr(A, K, Q_mpc, R_mpc);
L = dlqe(A, eye(n), C, B*B', D*D', B*D');

% 3. Matrici in Anello Chiuso (Verificare autovalori in cerchio unitario)
A_cl = [A - K*F, K*F; zeros(n,n), A - A*L*C];
B_cl = [B; B - L*D];

% Controllo di stabilita'
if max(abs(eig(A_cl))) >= 1
    error('Sistema in anello chiuso instabile.- Verifica le matrici. ');
end

% 4. Equazione di Lyapunov per la Covarianza di Stato
Sigma_xi = dlyap(A_cl, B_cl * B_cl');

% 5. Varianza dell'Input e Test Vincoli
M = [-F, F];
Sigma_u = M * Sigma_xi * M';
sigma_u = sqrt(diag(Sigma_u));

limite_superiore = uss + 3 * sigma_u;
limite_inferiore = uss - 3 * sigma_u;

violazione_max = limite_superiore > u_max;
violazione_min = limite_inferiore < u_min;
```

References

- [1] B. C. Levy and R. Nikoukhah, “Robust state space filtering under incremental model perturbations subject to a relative entropy tolerance,” *IEEE Trans. Autom. Control*, vol. 58, no. 3, pp. 682–695, Mar. 2013.
- [2] N. Anderson and Å. Björck, “A new high order method of regula falsi type for computing a root of an equation,” *BIT Numerical Mathematics*, vol. 13, no. 3, pp. 253–264, 1973.
- [3] S. M. L. Galdino, “A family of regula falsi root-finding methods,” 2011. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15854811>.